



G.T E C H

PROJECT SPECIFICATIONS

**Unreal Gameplay Architecture & Physics -
GTech 4**

PROJECT

FORMAT

Individual project

PROJECT CONTEXT

The objective of this first week is to familiarize you with Unreal Engine's gameplay architecture and the main systems used to build real-time physical interactions.

You will create an interactive physics sandbox in which a player can move through an environment and interact with various physical objects.

The goal is not to create a complete game, but to demonstrate your ability to build simple, clean, reusable, and technically sound gameplay systems.

OBJECTIVE

You must develop a playable sandbox in Unreal Engine featuring several interactions between the player and the environment.

Your prototype must notably demonstrate your understanding of:

- Unreal Engine's Gameplay Framework architecture;
- the use of Actors, Pawns, and Components;
- communication between different gameplay systems;
- input management;
- Rigid Body physics;
- collision and trigger management;
- the application of forces and impulses;
- the creation of player/environment interactions;
- the organization of a modular gameplay architecture;
- the use of simple debugging tools.

EXPECTED RESULT

Create an interactive physics sandbox

The player must be able to move freely within a test environment and experiment with different interactions involving physical objects.

The sandbox must offer several situations in which the player can manipulate and observe the behavior of the systems developed.

The work must nevertheless remain focused on gameplay programming and physics.

MANDATORY FEATURES

Player Movement

The prototype must include a controllable Character or Pawn with, at minimum:

- functional movement;
- camera control;
- a properly structured input system.

The movement system must allow the player to explore the sandbox comfortably.

Interaction System

The player must be able to detect and manipulate different elements of the environment.

You must implement a generic interaction system that allows multiple objects to react to the player without creating object-specific logic in the character for each object.

Interactions may notably use:

- collisions;
- overlaps;
- traces;
- interfaces;
- components;
- events.

You are free to choose the architecture, but you must be able to explain it.

Physical Objects

The sandbox must contain several objects using Unreal Engine's physics simulation.

Their behavior must allow you to experiment with, in particular:

- mass;
- gravity;
- collisions;
- friction;
- forces;
- impulses.

The various physics parameters must be used consistently with the proposed interactions.

Push / Pull / Launch

The player must have mechanics that allow direct interaction with physical objects.

At minimum, the prototype must provide the ability to:

Push — push an object or apply a force to it.

Pull — attract, move, or bring an object toward the player or a specified position.

Launch — propel an object using a force or an impulse.

These systems may be combined into a single mechanic or developed as several distinct interactions.

Collisions and Triggers

The sandbox must use several types of collisions and trigger zones.

You must demonstrate your ability to correctly distinguish and use:

- physical collisions;
- overlaps;
- triggers;
- responses to different Collision Channels.

The settings must be consistent with the expected behaviors.

EXPECTED ARCHITECTURE

Particular attention will be paid to the technical organization of the project.

You must avoid concentrating all logic in a single Blueprint.

The project must show a clear separation between:

Player → Interaction → Interactive Objects → Physics Systems → Environment

You are free to use Blueprint and/or C++ according to the needs of the system.

As far as possible, the systems developed must be:

- reusable;
- modular;
- readable;
- easy to modify;
- independent of the Level Blueprint.

You must be able to explain the role of the main classes used, as well as the communication between your systems.

INTRODUCTION TO CHAOS

The project must include an initial experiment with Chaos or Unreal Engine's related physics systems.

This experiment may remain simple.

The objective is to begin exploring the possibilities of the physics engine before studying them in greater depth the following week.

DEBUGGING

Your prototype must include at least one tool that helps visualize or understand how a gameplay system works.

For example:

- displaying a Trace;
- displaying the object currently detected;
- visualizing a direction or force;
- physics information;
- interaction-system state;
- relevant debug messages or information.

The objective is not to multiply the amount of on-screen information, but to demonstrate a development approach that makes it possible to quickly identify how a system behaves.

CONSTRAINTS

The project is completed individually.

The prototype must:

- be developed in Unreal Engine;
- be playable;
- use physics simulation;
- offer several player/environment interactions;
- include Push / Pull / Launch mechanics;
- use collisions and triggers;
- present a coherent gameplay architecture;
- include an initial use of or experiment with Chaos;
- include at least one debugging tool;
- remain simple enough to be completed and stabilized within the allotted time.

Technical quality takes priority over the quantity of content.

DELIVERABLES

At the end of the week, you must submit:

Unreal Project

The complete, functional project containing all source files required for evaluation.

Playable Prototype

The sandbox must be launchable and testable so that all developed mechanics can be experienced.

Short Technical Documentation

Concise documentation presenting:

- the overall project architecture;
- the main classes / Blueprints / Components;
- how the interaction system works;
- the physics systems developed;
- the main technical choices made.

Screenshots or diagrams may be used to make the explanations easier to understand.

AI Statement of Intent

If you used one or more artificial intelligence tools during the project, you must provide a statement of intent of at least one page detailing:

- the tools used;
- where AI was used;
- why it was used;
- how it was used;
- the benefits obtained;
- any limitations encountered.

Anything produced with the help of AI must remain understood, verified, tested, and explainable.

EVALUATION CRITERIA

The evaluation will mainly focus on the following elements:

Gameplay Architecture

Quality of project organization, separation of responsibilities, modularity, and consistency of communication between systems.

Interaction System

Quality and generic nature of the system that allows the player to interact with the environment.

Physics

Consistent use of collisions, Rigid Bodies, forces, impulses, and physical properties.

Push / Pull / Launch Mechanics

Functionality, stability, and technical quality of the different physical interactions.

Technical Quality

Readability of Blueprints/C++, asset organization, and ability to evolve the systems developed.

Debugging

Presence and relevance of tools used to analyze how the prototype works.

Experience

Clarity of interactions, quality of controls, and the player's ability to understand and experiment with the proposed systems.

Technical Mastery

Ability to precisely explain the architecture, the systems developed, and the choices made.



EDUCATIONAL AND PROFESSIONAL OBJECTIVES OF THE PROJECT

By the end of this first week, the student must be able to:

- Understand and use the fundamentals of Unreal Engine's Gameplay Framework.
- Build the initial technical foundations of an Unreal project intended to evolve over several weeks.
- Understand the roles and responsibilities of Actors, Pawns, and Components.
- Implement a clear, modular, and scalable gameplay architecture.
- Design a generic and reusable interaction system.
- Implement player inputs and movement consistent with the intended experience.
- Understand and apply the fundamental principles of real-time physics in Unreal Engine.
- Correctly configure Rigid Bodies, collisions, triggers, and collision responses.
- Manipulate physical objects through forces and impulses.
- Develop initial Push / Pull / Launch physical interaction mechanics.
- Properly organize communication between different gameplay systems in Blueprint and/or C++.
- Avoid tightly coupled architectures and favor reusable, maintainable systems.
- Implement simple debugging tools for observing and analyzing the behavior of the systems developed.
- Identify and correct the main issues related to interactions, collisions, and physical behaviors.
- Adopt a rapid prototyping approach: design, implement, test, analyze, and iterate.
- Learn to manage the scope of a production carried out under time constraints.
- Document and be able to explain the architecture and implementation choices made.
- Develop an initial professional approach to organizing an Unreal Engine project.
- Lay technical foundations robust enough to allow the same sandbox game to evolve over the following 8 weeks.

GRADING SYSTEM

Block 1 — Week 1: Unreal Gameplay Architecture & Physics

Knowledge

- Explain Unreal Engine's Gameplay Framework architecture and the role of the main classes: GameMode, PlayerController, Pawn/Character, Actor, and Component.
- Describe the different methods of communication between gameplay systems in Blueprint and/or C++.
- Explain how physics simulation generally works in Unreal Engine.
- Identify the role of collisions, Collision Channels, Overlaps, and Triggers.
- Explain the principles of forces, impulses, mass, gravity, and Rigid Body behavior.
- Understand the value of a modular and scalable gameplay architecture.
- Identify the role of the main gameplay and physics debugging tools.
- Describe the fundamental principles of Chaos and its integration into a gameplay system.

Practical Skills

- Build the technical foundations of the sandbox game intended to evolve over the 9 weeks.
- Configure a playable Character or Pawn with coherent inputs and movement.
- Design a generic and reusable interaction system.
- Create and configure objects that use physics simulation.
- Correctly configure collisions, Overlaps, Triggers, and Collision Channels.
- Implement physical interactions based on forces and impulses.
- Develop Push / Pull / Launch manipulation mechanics.
- Ensure clean communication between multiple gameplay systems.
- Integrate an initial experiment using Chaos or an associated physics system.
- Create simple tools to visualize and debug gameplay systems.
- Test and stabilize the prototype's different interactions.

- Properly organize the project, its folders, assets, Blueprints, and/or C++ classes.

Professional Attitudes / Skills

- Plan development according to the time available.
- Prioritize essential features and control the scope.
- Adopt a regular prototyping and iteration process.
- Test the systems developed regularly rather than waiting until the end of production.
- Analyze technical problems and methodically investigate their origin.
- Maintain clean organization so the project can evolve over the following weeks.
- Justify architecture and implementation choices.
- Document the project's important systems.
- Demonstrate autonomy in solving technical problems.
- Self-assess the quality of your work and identify areas for improvement.



EVALUATION GRID

Category	Evaluation Criterion	Points
Knowledge (Knowledge)	Explain Unreal Engine's Gameplay Framework architecture and the role of the main classes: GameMode, PlayerController, Pawn/Character, Actor, and Component.	10
	Describe the different communication methods between gameplay systems in Blueprint and/or C++.	10
	Explain the general operation of physics simulation in Unreal Engine.	10
	Explain the principles of forces, impulses, mass, gravity, and Rigid Body behavior.	10
	Understand the value of a modular and scalable gameplay architecture.	10
	Describe the fundamental principles of Chaos and its integration into a gameplay system.	10
Total Knowledge		60
Skills (Skills)	Structure the technical foundations of the sandbox game intended to evolve over the 9 weeks.	10
	Configure a playable Character or Pawn with coherent inputs and movement.	10
	Create and configure objects using physics simulation.	10

Category	Evaluation Criterion	Points
	Implement physical interactions based on forces and impulses.	10
	Integrate an initial experiment using Chaos or an associated physics system.	10
	Create simple tools to visualize and debug gameplay systems.	5
	Properly organize the project, its folders, assets, Blueprints, and/or C++ classes.	5
Total Skills		60
Professional Skills (Attitudes / Interpersonal Skills)	Plan development according to the available time.	5
	Prioritize essential features and control the scope.	5
	Adopt a regular prototyping and iteration process.	5
	Demonstrate autonomy in solving technical problems.	5
	Maintain a clean organization that allows the project to evolve over the following weeks.	5
	Justify technical choices, document the work, and identify areas for improvement.	5
Total Professional Skills		30
Grand Total		150

INSTRUCTOR

Name: RAO FERNANDES William

Title: PhD in Computer Science

Email: williamraofernandes@gmail.com

LinkedIn: <http://www.linkedin.com/in/william-rao-fernandes-011112b6>

BIOGRAPHY

PhD in Computer Science (CNAM Paris), specializing in difficulty in video games and its relationship with player motivation. A researcher and developer of a dynamic difficulty adaptation model, he is the author of several scientific publications on the subject.

For the past six years, he has worked as a freelance instructor in video games and artificial intelligence, and has taught at Gaming Campus since 2022, combining scientific expertise with practical teaching to train and inspire the next generation of industry professionals.